T7 Regresión lineal

Índice

1. Introducción
2. Modelo
3. Ajuste
4. Evaluación

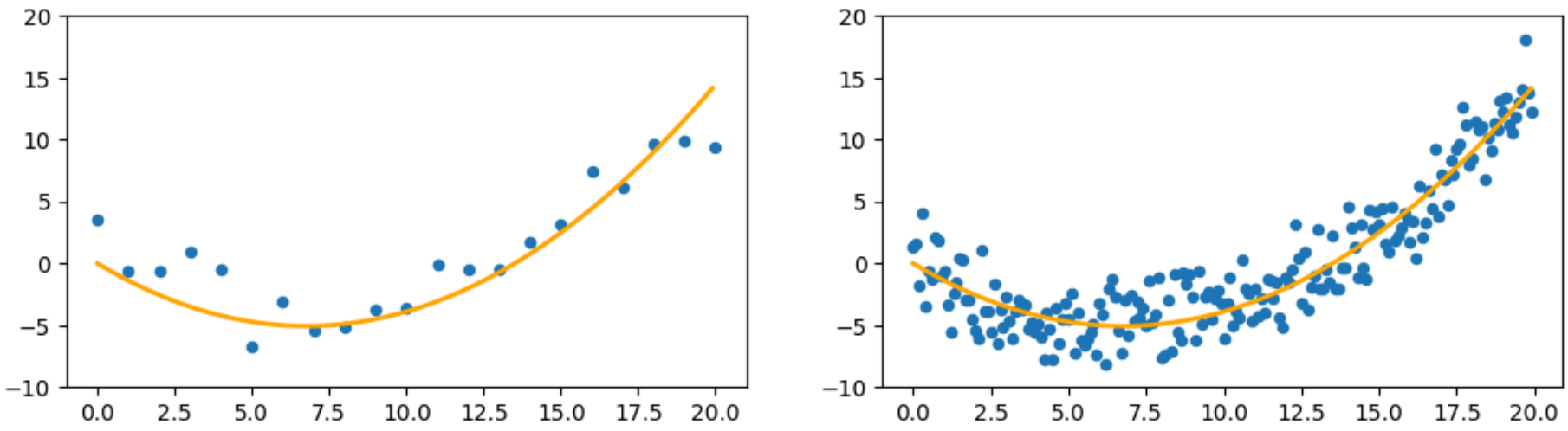
1 Introducción

Objetivo: introducir el modelo básico de regresión

Ejemplo: regresión polinómica (de grado 2) simple con datos sintéticos de entrenamiento (izq.) y test (der.)

$$f(x; \boldsymbol{w}) = w_0 + w_1 x + w_2 x^2 \quad \text{con} \quad w_0 = 0, \quad w_1 = -1.5 \quad \text{y} \quad w_2 = 1/9 = 0.11; \quad \sigma^2 = 4$$

In [ ]:
```
import numpy as np; np.random.seed(0); import matplotlib.pyplot as plt
w0 = 0; w1 = -1.5; w2 = 1/9; sigma = 2; f = lambda X: w0 + w1 * X + w2 * X * X
N = 21; X = np.linspace(0.0, 20, N); y = f(X) + np.random.normal(0, sigma, X.shape)
Xt = np.arange(0.0, 20, 0.1); yt = f(Xt) + np.random.normal(0, sigma, Xt.shape)
fig, axs = plt.subplots(1, 2, figsize=(12, 3))
ax = axs[0]; ax.set_ylim([-10, 20]); ax.scatter(X, y, s=20); ax.plot(Xt, f(Xt), c='orange', lw=2)
ax = axs[1]; ax.set_ylim([-10, 20]); ax.scatter(Xt, yt, s=20); ax.plot(Xt, f(Xt), c='orange', lw=2);
```



2 Modelo

Regresión lineal: normal condicional para regresión, $y \in \mathbb{R}$,

$$p(y \mid \boldsymbol{x}, \boldsymbol{\theta}) = \mathcal{N}(y \mid \mu, \sigma^2) \qquad \text{con} \qquad \mu = w_0 + \boldsymbol{w}^t \boldsymbol{x}$$

Notación homogénea: $\boldsymbol{w}$ absorbe el sesgo w_0

Regresión simple o múltiple: **simple** si la entrada es unidimensional; si no, **múltiple**

Regresión polinómica: $\boldsymbol{x}$ se procesa mediante un **extractor de características polinómico** ϕ

Extractor polinómico simple de grado D: $\boldsymbol{\phi}(x) = (1, x, x^2, \ldots, x^D)^t$

Regresión polinómica simple: para varianza σ^2 fija (regresión homocedástica)

$$p(y \mid x, \boldsymbol{\theta}) = \mathcal{N}(y \mid \mu, \sigma^2) \qquad \text{con} \qquad \mu = \boldsymbol{w}^t \boldsymbol{\phi}(x) = \sum_{d=0}^{D} w_d \, x^d$$

Estimador de Bayes: con pérdida cuadrática, $\ell_2(y - \pi(\boldsymbol{x})) = (y - \pi(\boldsymbol{x}))^2$, es la media a posteriori

$$\pi^*(\boldsymbol{x}) = \operatorname*{argmin}_{\pi(\boldsymbol{x})} \; R(y \mid \boldsymbol{x}) = \operatorname*{argmin}_{\pi(\boldsymbol{x})} \; \mathbb{E}_{p(y \mid \boldsymbol{x}, \boldsymbol{\theta})}[(y - \pi(\boldsymbol{x}))^2] = \mathbb{E}[y \mid \boldsymbol{x}] = \mu$$

Riesgo de Bayes teórico ($\boldsymbol{w}$ conocido): es la varianza

$$R^*(y \mid \boldsymbol{x}) = \mathbb{E}_{p(y \mid \boldsymbol{x}, \boldsymbol{\theta})}[(y - \pi^*(\boldsymbol{x}))^2] = \mathbb{E}_{p(y \mid \boldsymbol{x}, \boldsymbol{\theta})}[(y - \mu)^2] = \sigma^2$$

3 Ajuste

Neg-log-verosimilitud: de $\boldsymbol{\theta} = (\boldsymbol{w}^t, \sigma^2)^t$ respecto a N datos $\mathcal{D} = \{(\boldsymbol{x}_n, y_n)\}$ no normalizada

$$\mathrm{NLL}(\boldsymbol{\theta}) = -\sum_{n=1}^{N} \log p(y_n \mid \boldsymbol{w}^t \boldsymbol{x}_n, \sigma^2) = \frac{N}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_n (y_n - \boldsymbol{w}^t \boldsymbol{x}_n)^2$$

Residual sum of squares (RSS): equivalente a la NLL para minimizar en $\boldsymbol{w}$, con $\boldsymbol{y} = (y_1, \ldots, y_N)^t$

$$\mathrm{RSS}(\boldsymbol{w}) = \sum\nolimits_n (y_n - \boldsymbol{w}^t \boldsymbol{x}_n)^2 = \|\boldsymbol{y} - \mathbf{X}\boldsymbol{w}\|_2^2 = (\boldsymbol{y} - \mathbf{X}\boldsymbol{w})^t(\boldsymbol{y} - \mathbf{X}\boldsymbol{w})$$

Gradiente (transpuesto):

$$\frac{\partial\, \mathrm{RSS}(\boldsymbol{w})}{\partial \boldsymbol{w}} = 2\sum_n (\boldsymbol{w}^t \boldsymbol{x}_n - y_n)\boldsymbol{x}_n^t = 2(\mathbf{X}\boldsymbol{w} - \boldsymbol{y})^t \mathbf{X}$$

Ordinary least squares (OLS):

$$\hat{\boldsymbol{w}} = \mathbf{X}^\dagger \boldsymbol{y} \quad \text{donde} \quad \mathbf{X}^\dagger = (\mathbf{X}^t\mathbf{X})^{-1}\mathbf{X}^t \text{ es la pseudoinversa izquierda de } \mathbf{X}$$

Error cuadrático medio de los residuos:

$$\hat{\sigma}^2 = \operatorname*{argmin}_{\sigma^2} \ \mathrm{NLL}(\hat{\boldsymbol{w}}, \sigma^2) = \frac{1}{N}\sum_n (y_n - \hat{\boldsymbol{w}}^t \boldsymbol{x}_n)^2$$

Ejemplo: $p(y_n \mid x_n, m, c, \sigma^2) = \mathcal{N}(y_n \mid mx_n + c, \sigma^2),\ \mathcal{D} = \{(x_n, y_n)\} = \{(0,0), (1, 0.5), (1, 1.5), (2, 2)\}$

$$\boldsymbol{w} = (c, m)^t \qquad \mathbf{X} = [1, 0; 1, 1; 1, 1; 1, 2] \qquad \boldsymbol{y} = (0, 1/2, 3/2, 2)$$

$$\mathbf{X}^t\mathbf{X} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 1 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 4 & 4 \\ 4 & 6 \end{pmatrix}$$

$$(\mathbf{X}^t\mathbf{X})^{-1} = \frac{1}{8}\begin{pmatrix} 6 & -4 \\ -4 & 4 \end{pmatrix} = \begin{pmatrix} 3/4 & -1/2 \\ -1/2 & 1/2 \end{pmatrix}$$

$$(\mathbf{X}^t\mathbf{X})^{-1}\mathbf{X}^t = \begin{pmatrix} 3/4 & -1/2 \\ -1/2 & 1/2 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 2 \end{pmatrix} = \begin{pmatrix} 3/4 & 1/4 & 1/4 & -1/4 \\ -1/2 & 0 & 0 & 1/2 \end{pmatrix}$$

$$\hat{\boldsymbol{w}} = (\mathbf{X}^t\mathbf{X})^{-1}\mathbf{X}^t\boldsymbol{y} = \begin{pmatrix} 3/4 & 1/4 & 1/4 & -1/4 \\ -1/2 & 0 & 0 & 1/2 \end{pmatrix} \begin{pmatrix} 0 \\ 1/2 \\ 3/2 \\ 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$\hat{\boldsymbol{y}} = \mathbf{X}\hat{\boldsymbol{w}} = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 2 \end{pmatrix}$$

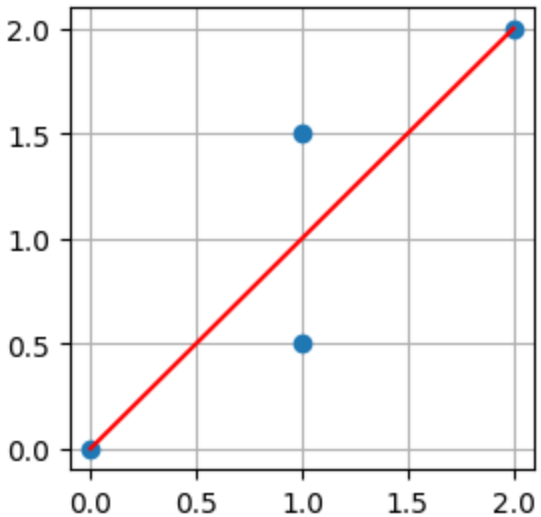
$$\mathrm{RSS}(\hat{\boldsymbol{w}}) = (\boldsymbol{y} - \hat{\boldsymbol{y}})^t(\boldsymbol{y} - \hat{\boldsymbol{y}}) = (0, 1/2, -1/2, 0)(0, 1/2, -1/2, 0)^t = 1/2$$

In [ ]:
```
import numpy as np; np.set_printoptions(formatter={'float': lambda x: "{0:0.2f}".format(x)})
X = np.array([[1,0],[1,1],[1,1],[1,2]]); y = np.array([0,.5,1.5,2]); eta = .05; w = np.zeros(2)
for i in range(100):
    w = w - eta * 2.0*X.T@(X@w-y)
print(w)
```

```
[0.00 1.00]
```

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt
X = np.array([0.0, 1.0, 1.0, 2.0]); N = len(X); Xh = np.c_[np.ones(N), X]
y = np.array([0.0, 0.5, 1.5, 2.0]); hw = np.linalg.inv(Xh.T @ Xh) @ Xh.T @ y
y_pred = Xh @ hw; RSS = np.square(y - y_pred).sum(); print(f'hw = {hw} RSS = {RSS}')
[m, c], [r], _, _ = np.linalg.lstsq(np.c_[X, np.ones(N)], y, rcond=None)
print(f'np.linalg.lstsq: hw = {hw} RSS = {RSS}')
fig = plt.figure(figsize=(3, 3)); plt.plot(X, y, 'o'); plt.plot(X, m*X + c, 'r'); plt.grid(True)
```

```
hw = [0. 1.] RSS = 0.5
np.linalg.lstsq: hw = [0. 1.] RSS = 0.5
```



4 Evaluación

Gráfica de residuos: si la entrada es unidimensional, evaluamos el modelo con $r_n = y_n - \hat{y}_n$ en función de x_n

Gráfica $\hat{y}_n$ vs y_n: si la entrada es multidimensional, evaluamos con $\hat{y}_n$ en función de y_n; diagonal óptima

Residual sum of squares: medida de calidad obvia

$$\mathrm{RSS}(\boldsymbol{w}) = \sum_n (y_n - \boldsymbol{w}^t \boldsymbol{x}_n)^2$$

Raíz del error cuadrático medio (RMSE): RSS normalizada

$$\mathrm{RMSE}(\boldsymbol{w}) = \sqrt{\frac{1}{N}\mathrm{RSS}(\boldsymbol{w})}$$

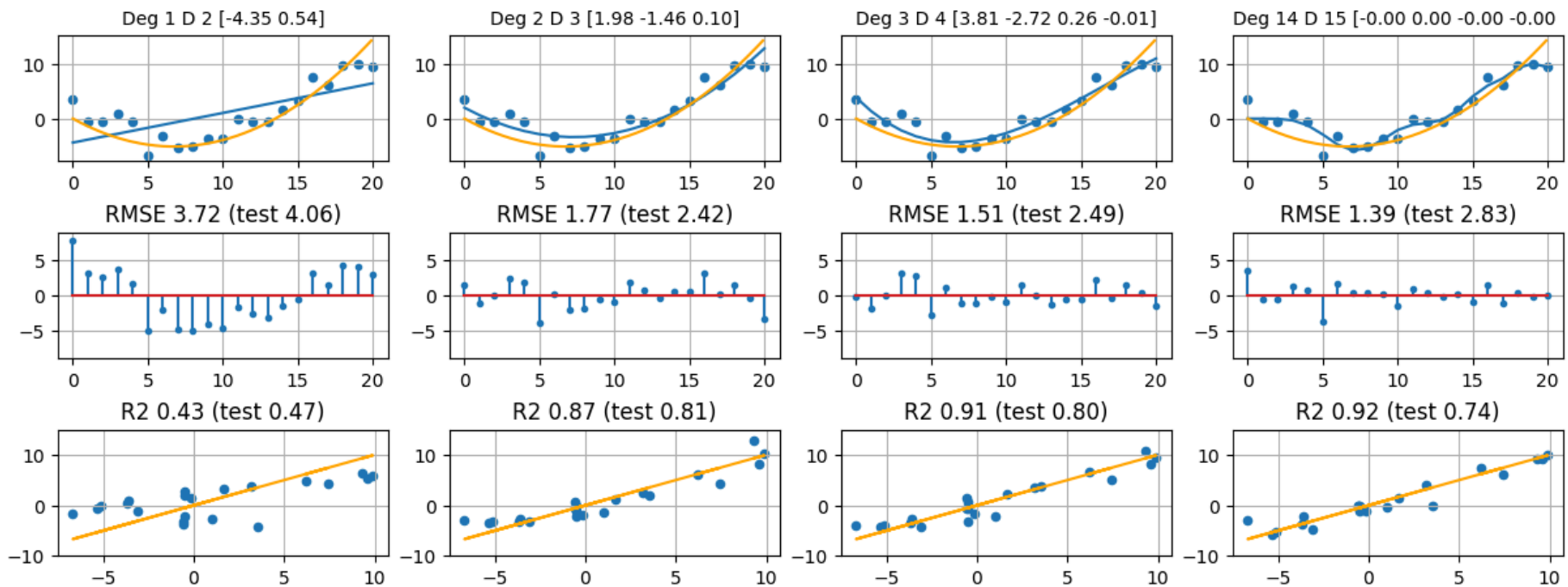
Suma de cuadrados total: suma de errores cuadráticos si siempre se predice la media empírica

$$\mathrm{TSS}(\boldsymbol{w}) = \sum_n (y_n - \bar{y})^2 \quad \text{con} \quad \bar{y} = \frac{1}{N}\sum_n y_n$$

Coeficiente de determinación: varianza de las predicciones en relación con predecir siempre la media empírica

$$R^2 = 1 - \frac{\mathrm{RSS}}{\mathrm{TSS}}$$

Interpretación de R^2: cuanto más próximo a 1 sea, mayor será la reducción de la varianza y mejor el ajuste

Ejemplo: $f(x;\boldsymbol{w}) = w_0 + w_1 x + w_2 x^2$ con $w_0 = 0,\ w_1 = -1.5$ y $w_2 = 1/9 = 0.11;\quad \sigma^2 = 4$

In [ ]:
```
import numpy as np; np.random.seed(0); import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures; from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error as RMSE, r2_score as R2
w0 = 0; w1 = -1.5; w2 = 1/9; sigma = 2; f = lambda X: w0 + w1 * X + w2 * X * X
N = 21; X = np.linspace(0.0, 20, N); y = f(X) + np.random.normal(0, sigma, X.shape)
Xt = np.arange(0.0, 20, 0.1); yt = f(Xt) + np.random.normal(0, sigma, Xt.shape); degs = [1, 2, 3, 14]
fig, axs = plt.subplots(3, len(degs), figsize=(12, 4.5), constrained_layout=True)
for i, deg in enumerate(degs):
    poly = PolynomialFeatures(degree=deg, include_bias=True); Xp = poly.fit_transform(X.reshape(-1, 1))
    lr = LinearRegression(fit_intercept=False).fit(Xp, y); y_pred = lr.predict(Xp)
    Xtp = poly.transform(Xt.reshape(-1, 1)); yt_pred = lr.predict(Xtp)
    ax = axs.flat[i]; ax.grid(); ax.scatter(X, y, s=20); ax.plot(X, y_pred)
    ax.plot(Xt, f(Xt), color='orange'); ax.set_title(f'Deg {deg} D {Xp.shape[1]} {lr.coef_!s:.24s}', fontsize=10)
    ax = axs.flat[i+len(degs)]; ax.grid(); ax.set_ylim(-9, 9); ax.stem(X, y - lr.predict(Xp), markerfmt='.')
    ax.set_title(f'RMSE {RMSE(y, y_pred):.2f} (test {RMSE(yt, yt_pred):.2f})')
    ax = axs.flat[i+2*len(degs)]; ax.grid(); ax.set_ylim(-10, 15); ax.scatter(y, y_pred, s=20)
    ax.plot(y, y, color='orange'); ax.set_title(f'R2 {R2(y, y_pred):.2f} (test {R2(yt, yt_pred):.2f})')
```